

Authentication & Authorization — Lecture Notes

1. Core Definitions

Term	Question it answers	Definition
Authentication	<i>Who are you?</i>	Mechanism to assign and verify an identity in a given context
Authorization	<i>What can you do?</i>	Mechanism to define and enforce permissions for an authenticated identity

These two are distinct and sequential — you must authenticate before you can authorize.

2. Historical Context of Authentication

Era	Mechanism	Principle
Pre-industrial	Village elder vouching; handshake deals	Implicit trust — human-based, couldn't scale
Medieval	Wax seals on documents	Something you possess — first "authentication token"; prone to forgery (first bypass attacks)
Industrial Revolution	Telegraph pass phrases	Something you know — shared static secrets; precursor to passwords
1961 — MIT CTSS	Digital passwords for multi-user mainframes	Stored in plain text → password printed accidentally → motivated secure storage
1970s	Hashing algorithms; Diffie-Hellman key exchange	Asymmetric cryptography (PKI); Kerberos introduced ticket-based auth (precursor to tokens)
1990s	MFA — Multi-Factor Authentication	Combines: something you know + something you have + something you are (biometrics)
21st century	OAuth 2.0, JWTs, Zero Trust, Passwordless (WebAuthn)	Cloud/API/mobile demands; modern frameworks
Future	Decentralized identity (blockchain), Behavioral	Quantum computers will break current RSA/PKI; post-quantum algorithms are being developed

Era	Mechanism	Principle
	biometrics, Post-Quantum cryptography	

Hashing = deterministic, one-way, fixed-length output regardless of input length. Same input always produces same hash. No way to reverse it back to plain text.

3. Three Key Components of Modern Auth

Before auth types: understand these three primitives that appear everywhere.

3.1 Sessions

HTTP is **stateless by design** — fine for early static web, but modern apps (e-commerce carts, login persistence) need memory.

Sessions solve this:

1. User logs in → server creates unique Session ID
2. Server stores Session ID + user data in persistent store (Redis / DB)
3. Session ID is sent to client as a cookie
4. Every subsequent request carries that cookie
5. Server looks up Session ID in store → retrieves user data
6. Session has an expiry; server creates a new one when it expires

Storage evolution:

- File-based → DB-backed → Distributed in-memory stores (Redis, Memcached)
- Redis preferred: in-memory = faster lookups than DB; survives server restarts

3.2 JWTs (JSON Web Tokens)

Why JWTs emerged: By mid-2000s, stateful sessions became costly for globally distributed systems — maintaining session data for millions of users, replicating across regions introduced latency.

JWT = self-contained, stateless token (formalized 2015).

Structure — 3 parts (Base64 encoded, dot-separated):

```
header.payload.signature
```

Part	Contents
Header	Metadata: signing algorithm (e.g., HS256)
Payload	User data: <code>sub</code> (user ID), <code>iat</code> (issued at), optional: <code>name</code> , <code>role</code>
Signature	Cryptographic signature using server's secret key — verifies integrity and origin

Verification: Server uses its secret key to verify the signature. If the payload is tampered with, the signature check fails → token rejected.

Advantages of JWTs:

Advantage	Detail
Stateless	No server-side storage needed; all data is in the token
Scalable	All servers share the secret key; any instance can verify any token
Portable	Base64-encoded, URL-friendly; can go in headers, cookies, local storage

Disadvantages:

Disadvantage	Detail
No revocation	Until expiry, a stolen JWT can't be invalidated (unless secret key is rotated — which logs out all users)
No real-time control	Can't immediately revoke a compromised account's access

Hybrid approach (mitigates above): Use JWTs normally, but maintain a **blacklist** in Redis. After JWT verification, check the blacklist. Downside: reintroduces a storage lookup — partially defeats statelessness. A common practical tradeoff.

Advice: For production systems, use an auth provider (Auth0, Clerk, etc.) — they handle token security, rotation, storage, algorithms, salting, and all edge cases.

3.3 Cookies

A mechanism for a server to **store a small piece of data in the client's browser** — and have it automatically sent back with every subsequent request to that server.

1. User authenticates → server sets a cookie (Set-Cookie header)
Cookie value = session ID or JWT token
2. Browser stores the cookie (scoped to that server's domain)

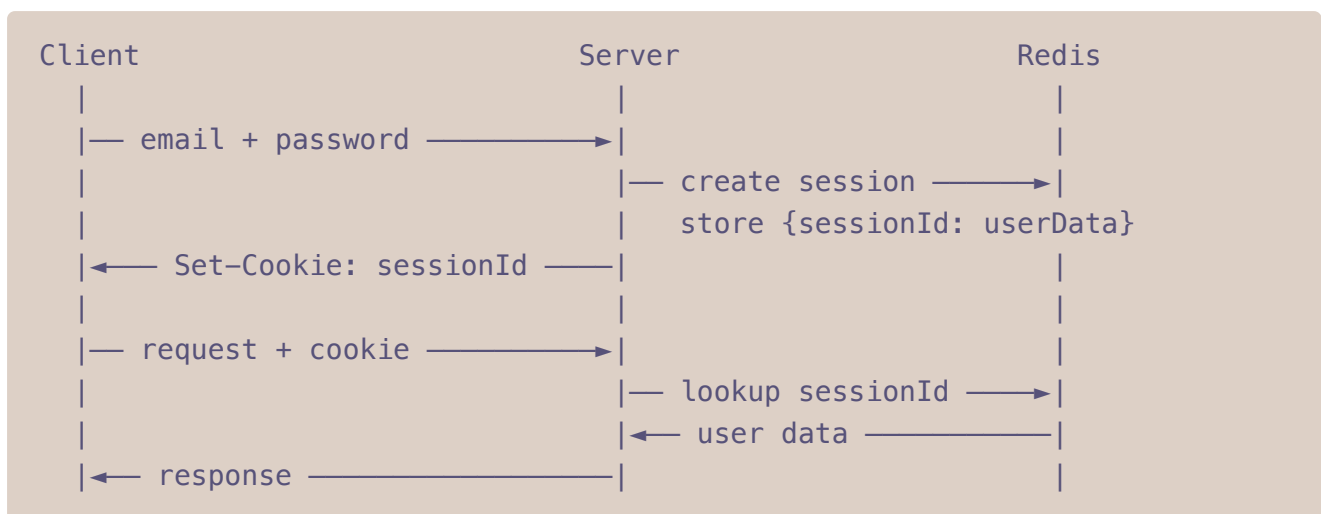
3. Browser automatically attaches cookie to all future requests to that server
4. Server reads cookie → validates the token/session

Security flags:

- **HttpOnly**: JavaScript cannot read the cookie — protects against XSS
- **Secure**: Cookie only sent over HTTPS
- Cookies set by one server are not visible to other servers — enforced by the browser

4. Types of Authentication

4.1 Stateful Authentication (Session-Based)

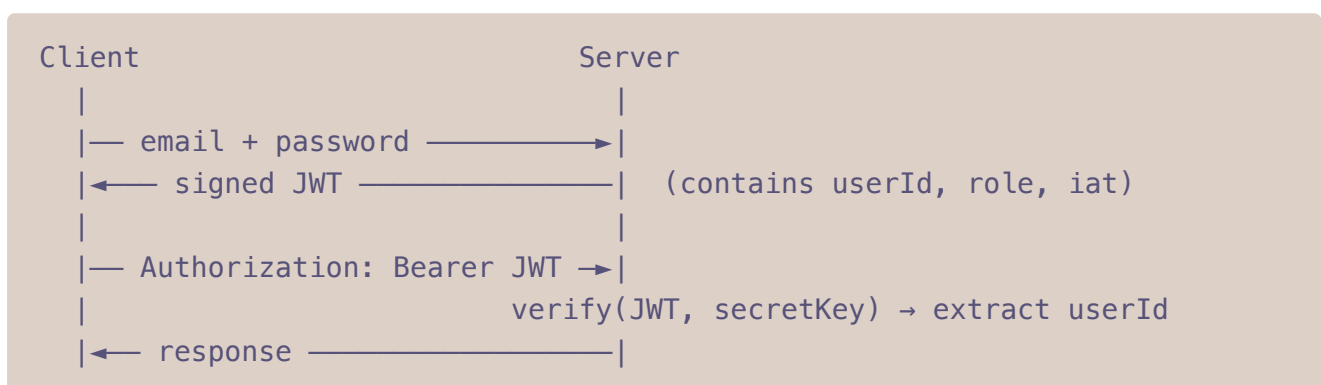


- Session ID is typically a cryptographically random string.
- Server stores state → centralized control → easy revocation.

Pros: Centralized control, real-time session visibility, easy revocation, suits web apps.

Cons: Storage overhead for millions of users; synchronization latency in distributed systems.

4.2 Stateless Authentication (JWT-Based)



- No storage lookup needed — JWT is self-validating.
- **401 Unauthorized** if token is missing/expired; **403 Forbidden** if token is valid but lacks permission.

Pros: Scalable, no session store, ideal for distributed architectures and mobile apps.

Cons: Token revocation is hard; compromised token valid until expiry.

4.3 API Key Authentication

Use case: Machine-to-machine (M2M) communication — no human in the loop.

Developer UI → Generate API Key → cryptographically random string

My Server — API key in header —> Third-party Server
(e.g., OpenAI)
Identifies: identity, quota, permissions

vs. Session/JWT auth: Those require a login form (human trigger). API keys are pre-shared secrets suitable for programmatic, server-to-server calls.

Advantages:

- Easy to generate (one click in a UI)
- Ideal for M2M — no login flow needed
- Scope-able: specific permissions + expiry dates per key

Common examples: OpenAI API, Google Maps API, Stripe API.

4.4 OAuth 2.0 + OpenID Connect (OIDC)

The delegation problem: As the internet grew, one platform needed to access resources from another (e.g., travel app scanning Gmail for flight tickets). Initial "solution" of sharing passwords was disastrous — full access, impossible to scope or revoke.

OAuth 1.0 (2007): Introduced token-sharing instead of password-sharing. Tokens have specific, limited permissions.

Four roles in OAuth:

Role	Who it is	Example
Resource Owner	The user	You
Client	App requesting access	Facebook
Resource Server	Server holding the data	Google (contacts)

Role	Who it is	Example
Authorization Server	Issues tokens after user consents	Google's auth server

OAuth 2.0 (2010): Simplified OAuth 1.0 (replaced cryptographic signatures with bearer tokens). Introduced multiple flows for different app types:

Flow	Use Case
Authorization Code Flow	Server-side apps (most common)
Implicit Flow	Browser-based apps — <i>now discouraged</i> (security risks)
Client Credentials Flow	M2M communication (no user involved)
Device Code Flow	Input-limited devices (Smart TVs, CLI tools)

OAuth 2.0 gap: Handles *authorization* (what can you do) but not *authentication* (who are you).

OpenID Connect / OIDC (2014): Built on top of OAuth 2.0, adds an **ID Token** (a JWT) containing the user's identity — name, email, profile picture, issuer, `iat`.

OIDC Flow (Authorization Code + ID Token):

1. Client redirects user to Authorization Server (e.g., Google)
2. User logs in, grants permissions
3. Auth server returns: authorization code + ID token
4. Client exchanges auth code → access token (+ ID token if not already received)
5. Client uses access token to access Resource Server on behalf of user
6. ID token used to authenticate the user into the client platform

"Sign in with Google/Facebook/Discord" uses OIDC behind the scenes. The platform gets your identity from Google without you creating another password.

5. When to Use Which

Scenario	Auth Type
Web app with user sessions	Stateful (session ID + Redis)
Scalable APIs, mobile apps, distributed systems	Stateless (JWT)
Third-party login ("Sign in with Google")	OAuth 2.0 + OIDC

Scenario	Auth Type
Server-to-server / programmatic API access	API Keys
Web app + mobile hybrid	Stateful for browser; Stateless JWT for mobile/API clients

6. Authorization — RBAC

Authorization = what a verified user is *allowed to do*. Managed via **RBAC (Role-Based Access Control)**.

How it works:

- Users are assigned **roles** at registration (e.g., `user`, `admin`, `moderator`).
- Roles are assigned **permissions** per resource (e.g., `read`, `write`, `delete`).
- On each request, the server deduces the user's role (from JWT payload or DB lookup) and checks whether the role has permission for the requested operation.
- No permission → `403 Forbidden`.

Roles:

```

user      → read notes, write notes, delete own notes
admin     → all of the above + access dead zone notes + manage users
moderator → read + write only

```

Request: DELETE /dead-zone/notes/123

```

→ extract role from JWT → role = "user"
→ "user" lacks permission for dead zone → 403 Forbidden

```

7. Security Best Practices

7.1 Generic Error Messages

Never expose specific failure reasons during authentication — attackers use them to narrow their attack surface.

❌ Specific (Dangerous)	✅ Generic (Safe)
"User not found"	"Authentication failed"
"Incorrect password"	"Authentication failed"
"Account locked"	"Authentication failed"

A specific message like "incorrect password" confirms the username was valid — now the attacker only needs to brute-force the password.

Rule: All authentication failure paths → same generic message.

7.2 Timing Attacks

How it works: Server response time leaks information.

```
Step 1: Find user in DB          → fast (simple lookup)
Step 2: Check if account locked → fast
Step 3: Hash + compare password → slow (hashing is expensive)

Invalid username → fails at Step 1 → fast response (~50ms)
Valid username, wrong password → fails at Step 3 → slower response
(~250ms)

Attacker measures response time → deduces whether username exists
```

Defenses:

1. **Constant-time comparison functions** — cryptographically safe functions that take the same time regardless of input; don't short-circuit on mismatch.
 2. **Artificial delay** — even on early failure (wrong username), simulate the full response time (e.g., `setTimeout` in Node.js, `time.Sleep` in Go) so all failure paths take the same time.
-

Quick Revision Checklist

- ☐ Authentication = who are you; Authorization = what can you do
- ☐ Sessions: server-side state in Redis; session ID sent as cookie; stateful
- ☐ JWT: 3 parts (header, payload, signature); stateless; signed with secret key; `sub`, `iat`, `role` fields
- ☐ JWT pros: stateless, scalable, portable; cons: hard to revoke, no real-time control
- ☐ Cookies: server stores data in browser; auto-sent with requests; `HttpOnly` + `Secure` flags
- ☐ Stateful auth: session ID + Redis lookup; easy revocation; higher storage cost
- ☐ Stateless auth: JWT verified with secret key; no lookup; hard to revoke
- ☐ API keys: M2M communication; scoped permissions; no human login flow
- ☐ OAuth 2.0: authorization delegation via tokens; 4 flows; solves password-sharing problem
- ☐ OIDC: adds ID token (JWT) on top of OAuth 2.0; enables "Sign in with Google"
- ☐ RBAC: roles → permissions; role deduced per request; no permission → 403

- ☐ Security: always use generic auth error messages; equalize response times to prevent timing attacks
- ☐ Production: use an auth provider (Auth0, Clerk) unless you have a very good reason not to